

Проектний практикум

Звіт з лабораторної роботи 1

Ініціювання командного IT проекту

Проект EasyStock

Вебзастосунок для обліку товарів і складських операцій

Виконав: Бондаренко Сергій

Група: ЗІПЗс

Форма виконання: індивідуальна

Дата: 05.10.2026

Усю роботу над проектом і цією лабораторною роботою я виконав самостійно. Я поєднував ролі автора ідеї, аналітика, проєктувальника, розробника, тестувальника та автора документації.

Репозиторій GitHub: <https://github.com/chaek438/EasyStock>

Перевірена версія main: 8e8a046cee04

Ролі «комірник», «менеджер» і «адміністратор» описують користувачів застосунку. Вони не є учасниками розробки.

Мета роботи та статут проєкту

Я обґрунтував вибір EasyStock, визначив користувачів, SMART мету, критерії успіху, межі MVP та порядок індивідуальної роботи. Підготовлений застосунок реалізує наскрізний складський сценарій; автоматизовані перевірки основної логіки пройдено.

Статут проєкту наведено на цій і наступній сторінках. Єдиний виконавець і відповідальний за всі артефакти — Бондаренко Сергій, група ЗІПЗс.

Проблема та цінність

Цільовий контекст — невеликий магазин або склад, у якому комірник приймає і списує товар, а менеджер перевіряє наявність перед продажем. Розрізнені таблиці та ручні підрахунки можуть створювати неузгоджені залишки, зайві уточнення і складність встановлення автора зміни. Це робоча гіпотеза навчального проєкту, а не результат вимірювання конкретного підприємства.

EasyStock має надати єдиний каталог, пошук за назвою або SKU, контроль кількості й журнал операцій. Користувач бачить результат у тому самому процесі, а залишок змінюється разом із записом історії.

SMART мета

До 05.10.2026 підготувати працездатний MVP EasyStock, у якому авторизований комірник може знайти товар, провести надходження, виконати допустиме списання і перевірити історію; надмірне списання має відхилятися без зміни даних. Технічне приймання — успішна production збірка та проходження всіх перевірок критичних правил у pytest.

Метрика	Початковий стан	Ціль і перевірка	Стан
Час пошуку товару людиною	Не виміряно	≤ 30 с; 5 повторів пошуку за SKU у свіжій демобазі	Потрібен людський прогін
Цілісність складських операцій	До тестування немає доказу	0 порушень у позитивних, негативних і конкурентних сценаріях	Тести пройдено
Час відповіді API	До тестування не виміряно	$p95 \leq 2$ с; 30 запитів за 500 товарів	У TESTING.md: 0,0037 с

Локальний p95 вимірювався через Flask test client. Він не вимірює затримку мережі чи час дій користувача. На перевірній версії GitHub повторно пройдено 39 тестів і production збірку.

Статут проєкту та межі продукту

Цільова група і зацікавлені сторони

Сторона	Потреба	Вплив	Порядок залучення
Комірник	Коректно проводити операції та бачити результат	Високий	Змодельована сесія і сценарій перевірки
Менеджер	Швидко знаходити товар і актуальний залишок	Середній	Змодельована сесія; майбутній usability прогін
Адміністратор	Керувати каталогом та доступом	Високий	Самоперевірка прав і довідників
Власник малого бізнесу	Прозорий облік і контроль дефіциту	Високий	Перевірка продуктових метрик у майбутньому
Викладач	Оцінити артефакти й обґрунтування рішень	Високий	Посилання на GitHub і захист
Бондаренко Сергій	Реалізувати та пояснити проєкт	Високий	Усі ролі виконую самостійно

Ядро MVP

Вхід → каталог і пошук → картка товару → надходження або списання → оновлений залишок → історія → вихід. Мінімальний каталог підтримує створення товару та базові довідники. Доступ контролюється сервером для трьох ролей застосунку.

Поза початковою межею MVP

CSV обмін, сканування штрихкодів, розширена аналітика, мобільний застосунок і зовнішні повідомлення не є умовою приймання ядра. У поточній версії додатково реалізовано редагування товарів, низькі залишки та керування користувачами; це розширення початкового плану.

Обмеження і критерії готовності

Навчальний локальний вебзастосунок; SQLite на постійному диску; кількість товару — ціле число; один розробник. Критерій технічної готовності: інсталяція за README, демонстраційний сценарій, 39 успішних тестів, збірка React і доступний GitHub.

Методичка передбачає командну роботу. Проєкт фактично виконано мною одноосібно; індивідуальний формат і заміну взаємної перевірки самоперевіркою потрібно погодити з викладачем. Таке погодження у цьому звіті не заявляю.

Вибір проблеми та відповідальність

Я порівняв три навчальні варіанти. Оцінки є моєю експертною оцінкою за шкалою 1–5, а не статистикою користувачів. Вищий бал означає кращу придатність; для ризику 5 означає нижчий ризик.

Варіант	Користь	Перевірка	Виконання	Ризик	Сума
FixFlow облік ремонтів	4	3	4	3	14
StudyHub навчальна платформа	4	3	4	3	14
EasyStock складський облік	5	4	5	4	18

Я обрав EasyStock, тому що правила залишку можна перевірити однозначно, а навчальні дані не потребують персональних або виробничих записів. Доступ до реальних працівників не підтверджено, тому моделювання придатне для формування вимог, але не доводить ринкову користь.

Індивідуальна матриця відповідальності

R означає виконання, А — відповідальність за результат. Оскільки роботу виконую один, в усіх внутрішніх завданнях R і А поєдную я. Викладач є стороною навчального оцінювання, а не учасником розробки.

Артефакт або дія	Поєднана роль	R і А
Статут, цілі, пріоритети	Автор ідеї та керівник проєкту	Бондаренко Сергій
Персони, backlog, критерії	Бізнес аналітик	Бондаренко Сергій
Архітектура, модель даних, ADR	Проектувальник	Бондаренко Сергій
API, React, база даних	Розробник	Бондаренко Сергій
Тести і самоперевірка	Тестувальник	Бондаренко Сергій
GitHub, звіти, захист	Автор документації	Бондаренко Сергій

Мій особистий внесок підтверджується кодом і документацією у репозиторії, результатами тестів та поясненням рішень на захисті. Один commit у main не дає детального журналу етапів роботи; надалі потрібні змістовні commits за завершеними змінами.

Правила роботи та план завершення

Я сформулював індивідуальні робочі правила замість угоди між кількома учасниками. Вони регулюють мою роботу та майбутню комунікацію з викладачем; підписаної командної угоди немає.

Правило	Як застосовую
Єдине джерело версії	Код, інструкції та артефакти зберігаю у GitHub; у звітах вказую версію.
Фіксація завдань	Кожне завдання повинне мати результат, статус, критерій завершення і виконавця Бондаренко Сергій.
Комунікація	Планую відповідати на навчальні уточнення протягом 24 годин у робочі дні.
Зміна пріоритетів	Спочатку забезпечую цілісність MVP; рішення і мотив записую у backlog або ADR.
Перевірка змін	Для бізнес правил запускаю відповідні тести; для інтерфейсу перевіряю основний і помилковий сценарії.
Готовність зміни	Код працює, дані узгоджені, інструкція відповідає реалізації, результат зафіксований.

Завдання для завершення подання

Завдання	Критерій завершення	Статус	Виконавець
Перевірити застосунок	39 тестів і збірка PASS	Виконано	Бондаренко Сергій
Підготувати 3 звіти	Є вимоги, діаграми, прототип і GitHub	Виконано у цих звітах	Бондаренко Сергій
Очистити Git від БД і кешу	БД, WAL, SHM і рус не відстежуються	До подання	Бондаренко Сергій
Додати ignore і оновити README	Є .gitignore, актуальні посилання й автор	До подання	Бондаренко Сергій
Створити дошку GitHub	Є доступне викладачу посилання і задачі	До подання	Бондаренко Сергій
Перевірити браузер і зручність	Записані результати і час пошуку	До фінального приймання	Бондаренко Сергій

Ця таблиця є планом, а не доказом створеної GitHub Projects дошки. У перевіреному репозиторії Issues відсутні, посилання на дошку не наведено. Конкретний навчальний дедлайн для невиконаних пунктів має визначити викладач.

Перевірка репозиторію GitHub

<https://github.com/chaek438/EasyStock>

Стан на 05.10.2026: репозиторій public, гілка main, commit 8e8a046cee04, 57 відстежуваних файлів. GitHub доступний за посиланням без запрошення викладача. Public означає, що репозиторій можуть переглядати й інші користувачі.

Перевірка	Фактичний результат	Оцінка
Структура	backend, frontend, database, docs, tests, README і стартові скрипти у корені	Коректна
Тести API і правил	39 passed, повторна перевірка клону, 25,43 с	Пройдено
Production збірка	Vite завершив збірку React без помилки	Пройдено
.gitignore	У main файл відсутній	Додати
Робочі дані і кеш	Tracked: .db, .db-wal, .db-shm та 7 рус файлів	Прибрати з Git
Інструкції GitHub	Є YOUR_REPOSITORY_URL і згадки, що публікацію ще треба виконати	Оновити
Історія main	1 commit Add files via upload; старі commits лише у git.bundle	Поліпшити наступні зміни
Issues і дошка	Issues порожні; посилання на реальну дошку немає	Підготувати

Необхідні виправлення оформлення

Додати .gitignore для .venv, node_modules, __pycache__, *.рус, *.db, *.db-wal, *.db-shm і локального .env. Уже відстежувані файли потрібно окремо прибрати з індексу Git; саме додавання .gitignore цього не виконує. Локальна база має створюватися зі схеми та demo seed.

У README вказати «Виконав: Бондаренко Сергій», справжнє посилання і стан дошки. У docs/Git.md замінити placeholder; старі твердження про відсутність публікації привести до фактичного стану.

Висновок перевірки: основна реалізація та структура працездатні. Оформлення GitHub і окремі навчальні докази потребують завершення. Віддалений репозиторій під час перевірки не змінювався.

Короткий захист і висновок

Текст виступу до трьох хвилин

Я, Бондаренко Сергій, самостійно розробив EasyStock — навчальний вебзастосунок складського обліку. Проект розв'язує задачу узгодження каталогу, залишку та історії операцій для невеликого магазину. Я обрав цю тему серед обліку ремонтів, навчальної платформи та складського обліку, оскільки складські правила можна перевірити точно.

Ядро MVP — вхід, пошук товару, надходження, списання й перегляд історії. У демонстраційній базі товар SSD-480 має 17 одиниць. Надходження 20 змінює залишок на 37; списання 40 відхиляється; списання 3 залишає 34. Автор і час успішної операції потрапляють до журналу. Менеджер може переглядати дані, але не проводити складські операції.

Я використав React, Flask, SQLAlchemy та SQLite. Залишок і журнал зберігаються однією транзакцією, а умовне SQL списання захищає від надмірного списання за одночасних запитів. На версії GitHub пройдено 39 тестів і production збірку. Я також підготував вимоги, діаграми, модель даних і прототип. Перед остаточним поданням потрібно завершити очищення GitHub, дошку завдань та людську перевірку зручності.

Висновок

Я сформував узгоджену основу проекту і перевінив його реалізоване ядро. Усі проектні ролі та всі роботи виконав самостійно. Головний технічний ризик — цілісність залишків — перевірено тестами; час пошуку людиною та браузерна сумісність залишаються окремими критеріями приймання.

Використані матеріали

Методичні матеріали викладача «Ініціювання командного ІТ проекту»: розгорнутий текст на 23 сторінках і презентація на 45 сторінках.

Код і документація проекту: <https://github.com/chaek438/EasyStock>

[Перевірений commit 8e8a046cee04](#)

[Результати технічних перевірок](#)

Відповіді на контрольні запитання

1 Чому проблема придатна для практикуму

Є конкретні ролі, наскрізний процес, сутності та правила з вимірюваним результатом; демонстрація можлива на безпечних навчальних даних.

2 Що є ядром MVP

Вхід → пошук → складська операція → актуальний залишок → запис історії. Відхилення надмірного списання належить до того самого ядра.

3 Хто визначає пріоритети

Я визначаю продуктові пріоритети самостійно і фіксую їх у backlog. Навчальні критерії та допустимість індивідуального формату оцінює викладач.

4 Як підтверджується особистий внесок

Усі артефакти мають одного виконавця — мене. Доказами є код, звіти, результати тестів і пояснення рішень; наступні зміни варто фіксувати окремими commits.

5 Чим problem statement відрізняється від функцій

Він визначає користувача, контекст, втрату і бажану зміну. Список функцій описує спосіб реалізації цієї зміни.

6 Яка метрика доводить користь

Час знаходження потрібного товару людиною. Технічна швидкодія API допоміжна; ціль ≤ 30 с ще потрібно виміряти.

7 Який ризик є найбільшим

Неузгодженість залишку й історії за помилки або конкуренції. Його знижують транзакція, умовний UPDATE, обмеження БД та тести.

8 Чому .gitignore не очищує історію

Він впливає на додавання невідстежуваних файлів. Файл у старому commit залишається там; секрет, якщо такий був, потрібно відкликати, а історію очищувати окремо.